

COMS 4776 Lecture 15: Variational Autoencoders

Richard Zemel

Overview

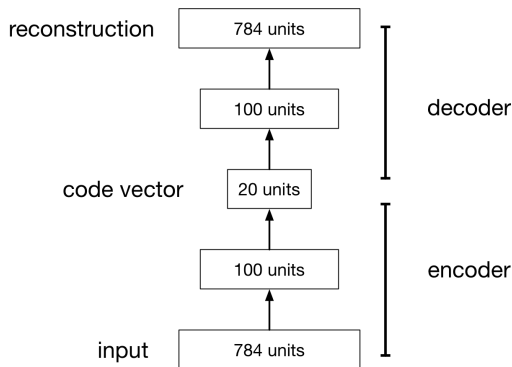
Three modern approaches to generative modeling:

- Autoregressive models
- Generative adversarial networks
- Variational autoencoders (this lecture)

All three approaches have different pros and cons.

Autoencoders

- An **autoencoder** is a feed-forward neural net whose job it is to take an input $\mathbf{x}$ and predict $\mathbf{x}$.
- To make this non-trivial, we need to add a **bottleneck layer** whose dimension is much smaller than the input.



Autoencoders

Why autoencoders?

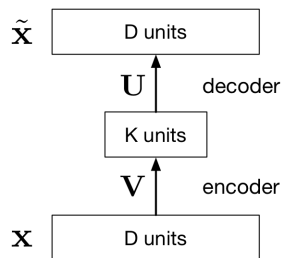
- Map high-dimensional data to two dimensions for visualization
- Compression (i.e. reducing the file size)
- Learn abstract features in an unsupervised way so you can apply them to a supervised task
 - Unlabeled data can be much more plentiful than labeled data
- Learn a semantically meaningful representation where you can, e.g., interpolate between different images.

Principal Component Analysis (optional)

- The simplest kind of autoencoder has one hidden layer, linear activations, and squared error loss.

$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2$$

- This network computes $\tilde{\mathbf{x}} = \mathbf{U}\mathbf{V}\mathbf{x}$, which is a linear function.
- If $K \geq D$, we can choose $\mathbf{U}$ and $\mathbf{V}$ such that $\mathbf{U}\mathbf{V}$ is the identity. This isn't very interesting.
- But suppose $K < D$:
 - $\mathbf{V}$ maps $\mathbf{x}$ to a K -dimensional space, so it's doing dimensionality reduction.
 - The output must lie in a K -dimensional subspace, namely the column space of $\mathbf{U}$.



Principal Component Analysis (optional)

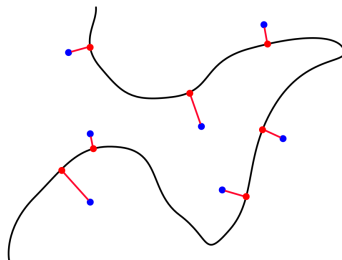
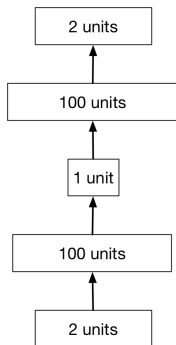
- Linear autoencoders with squared error loss are equivalent to Principal Component Analysis (PCA).
- Two equivalent formulations:
 - Find the subspace that minimizes the reconstruction error.
 - Find the subspace that maximizes the projected variance.
- The optimal subspace is spanned by the dominant eigenvectors of the empirical covariance matrix.



“Eigenfaces”

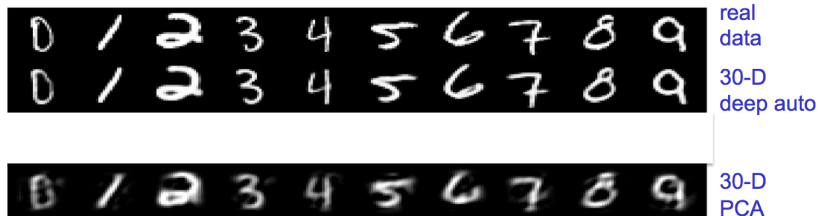
Deep Autoencoders

- Deep nonlinear autoencoders learn to project the data, not onto a subspace, but onto a nonlinear **manifold**
- This manifold is the image of the decoder.
- This is a kind of **nonlinear dimensionality reduction**.



Deep Autoencoders

- Nonlinear autoencoders can learn more powerful codes for a given dimensionality, compared with linear autoencoders (PCA)



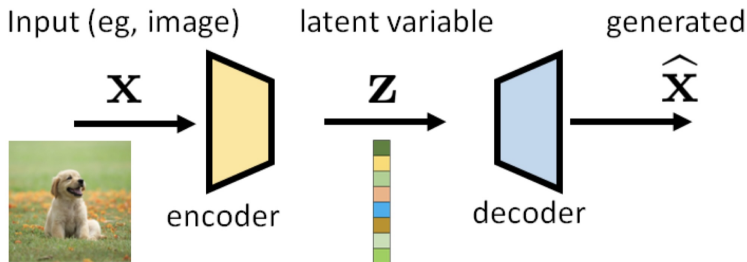
Deep Autoencoders

- Some limitations of autoencoders
 - They're not generative models, so they don't define a distribution
 - How to choose the latent dimension?

The Variational Autoencoder (VAE)

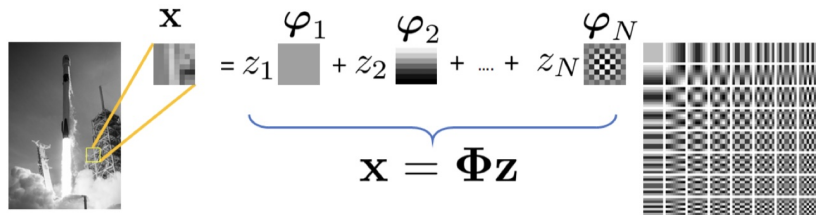
Three main components

- 1 Encoder: extracts meaningful representation for input $\mathbf{x}$
- 2 Latent variable $\mathbf{z}$: preserves important content of $\mathbf{x}$, and serves as seed to create which $\hat{\mathbf{x}}$ can be generated
- 3 Decoder: generates estimate of input ($\hat{\mathbf{x}}$) from latent variable



Latent Representation

- Aim is to concisely encode image/patches
- Analogy from JPEG compression: Discrete Cosine Transform uses basis functions to encode image patches
- Project $\mathbf{x}$ to obtain latent $\mathbf{z}$: coefficient vector $\{z_1, z_2, \dots, z_N\}$
- Decode using inverse transform


$$\mathbf{x} = z_1 \varphi_1 + z_2 \varphi_2 + \dots + z_N \varphi_N$$
$$\mathbf{x} = \Phi \mathbf{z}$$

- variational: from calculus of variations, which studies optimization over functions
- searching for the optimal probability distributions to describe $p(\mathbf{x})$ and $p(\mathbf{z})$
- consider relevant distributions
 - 1 $p(\mathbf{x})$: trying to estimate, to determine membership, draw samples
 - 2 $p(\mathbf{z})$: typically assume zero-mean unit-variance Gaussian: linear transforms remain Gaussian; generate any distribution by mapping through sufficiently complicated function
 - 3 $p(\mathbf{z}|\mathbf{x})$: conditional distribution associated with encoder, describes likelihood of $\mathbf{z}$ when given $\mathbf{x}$
 - 4 $p(\mathbf{x}|\mathbf{z})$: conditional distribution associated with decoder

Example: Gaussian Mixture Model

- Consider $\mathbf{X}$ distributed as a Gaussian mixture, $z \in \{1, \dots, K\}$ denotes cluster identity, $p_Z(k) = P[Z = k] = \pi_k$ for $k = 1, \dots, K$
- $p_{\mathbf{X}|Z}(\mathbf{x}|k) = \mathcal{N}(\mathbf{x}|\mu_k, \sigma_k^2 \mathbf{I})$
- Marginal of $\mathbf{x}$:

$$p_{\mathbf{X}}(\mathbf{x}) = \sum_{k=1}^K p_{\mathbf{X}|Z}(\mathbf{x}|k) p_Z(k) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \sigma_k^2 \mathbf{I})$$

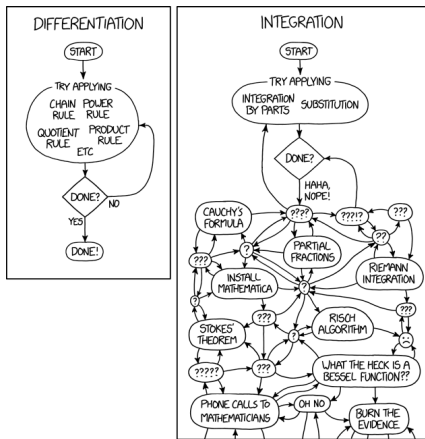
- Encoder: determine which component input $\mathbf{X}$ belongs to
- Decoder: Given latent $z \in \{1, \dots, K\}$, return sample from $p_{\mathbf{X}|Z}(\mathbf{x}|k) = \mathcal{N}(\mathbf{x}|\mu_k, \sigma_k^2 \mathbf{I})$

Generalizing to VAE

- But want to learn representations beyond mixtures with known means and variances
- To find good encoders and decoders, need to find good conditional distributions $p(\mathbf{X}|\mathbf{Z})$ and $p(\mathbf{Z}|\mathbf{X})$
- In this more general case, we need to estimate the data distribution
- Also called the *evidence*: the log marginal likelihood of observed data under the model, obtained by integrating over all possible values of the latent variables $\mathbf{z}$

Evidence

- At least $p(\mathbf{x}) = \int p(\mathbf{z})p(\mathbf{x} | \mathbf{z}) d\mathbf{z}$ is well-defined, but how can we compute it?
- Integration, according to XKCD:



- At least $p(\mathbf{x}) = \int p(\mathbf{z})p(\mathbf{x} | \mathbf{z}) d\mathbf{z}$ is well-defined, but how can we compute it?
 - $p(\mathbf{x} | \mathbf{z})$ depends on the decoder function $G_{\theta}(\mathbf{z})$
 - This is a very complicated function, so there's no hope of finding a closed form.
- Instead, we will try to maximize a lower bound on $\log p(\mathbf{x})$, known as the *ELBO*, or evidence lower bound.
 - The math is essentially the same as for the EM algorithm.

Variational Inference

- We obtain the lower bound using **Jensen's Inequality**: for a convex function h of a random variable X ,

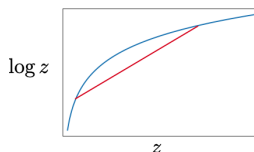
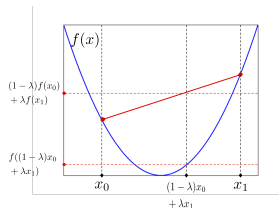
$$\mathbb{E}[h(X)] \geq h(\mathbb{E}[X])$$

Therefore, if h is **concave** (i.e. $-h$ is convex),

$$\mathbb{E}[h(X)] \leq h(\mathbb{E}[X])$$

- The function $\log z$ is concave. Therefore,

$$\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$$



Variational Inference

- Suppose we have some distribution $q(\mathbf{z}|\mathbf{x})$. (We'll see later where this comes from.)
- We use Jensen's Inequality to obtain the lower bound.

$$\begin{aligned}\log p(\mathbf{x}) &= \log \int p(\mathbf{z}) p(\mathbf{x}|\mathbf{z}) d\mathbf{z} \\ &= \log \int q(\mathbf{z}|\mathbf{x}) \frac{p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} p(\mathbf{x}|\mathbf{z}) d\mathbf{z} \\ &\geq \int q(\mathbf{z}|\mathbf{x}) \log \left[\frac{p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} p(\mathbf{x}|\mathbf{z}) \right] d\mathbf{z} \quad (\text{Jensen's Inequality}) \\ &= \mathbb{E}_q \left[\log \frac{p(\mathbf{z})}{q(\mathbf{z})} \right] + \mathbb{E}_q [\log p(\mathbf{x}|\mathbf{z})]\end{aligned}$$

- We'll look at these two terms in turn.

Variational Inference

- The first term we'll look at is $\mathbb{E}_q [\log p(\mathbf{x}|\mathbf{z})]$
- Since we assumed a Gaussian observation model,

$$\begin{aligned}\log p(\mathbf{x}|\mathbf{z}) &= \log \mathcal{N}(\mathbf{x}; G_{\theta}(\mathbf{z}), \eta \mathbf{I}) \\ &= \log \left[\frac{1}{(2\pi\eta)^{D/2}} \exp \left(-\frac{1}{2\eta} \|\mathbf{x} - G_{\theta}(\mathbf{z})\|^2 \right) \right] \\ &= -\frac{1}{2\eta} \|\mathbf{x} - G_{\theta}(\mathbf{z})\|^2 + \text{const}\end{aligned}$$

- So this term is the expected squared error in reconstructing $\mathbf{x}$ from $\mathbf{z}$. We call it the **reconstruction term**.

Variational Inference

- The second term is $\mathbb{E}_q \left[\log \frac{p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} \right]$.
- This is just $-\text{D}_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$, where D_{KL} is the **Kullback-Leibler (KL) divergence**

$$\text{D}_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \triangleq \mathbb{E}_q \left[\log \frac{q(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right]$$

- KL divergence is a widely used measure of distance between probability distributions, though it doesn't satisfy the axioms to be a distance metric.
- More details in tutorial.
- Typically, $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Hence, the KL term encourages q to be close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Variational Inference

- Hence, we're trying to maximize the **variational lower bound**, or **variational free energy**:

$$\log p(\mathbf{x}) \geq \mathcal{F}(\boldsymbol{\theta}, q) = \mathbb{E}_q [\log p(\mathbf{x}|\mathbf{z})] - D_{\text{KL}}(q\|p).$$

- The term “variational” is a historical accident: “variational inference” used to be done using variational calculus, but this isn't how we train VAEs.
- We'd like to choose q to make the bound as tight as possible.
- It's possible to show that the gap is given by:

$$\log p(\mathbf{x}) - \mathcal{F}(\boldsymbol{\theta}, q) = D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}|\mathbf{x})).$$

Therefore, we'd like q to be as close as possible to the posterior distribution $p(\mathbf{z}|\mathbf{x})$.

- Let's think about the role of each of the two terms.
- The reconstruction term

$$\mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] = -\frac{1}{2\sigma_{\text{dec}}^2}\mathbb{E}_q[\|\mathbf{x} - G_{\theta}(\mathbf{z})\|^2] + \text{const}$$

is minimized when q is a **point mass** on

$$\mathbf{z}_* = \arg \min_{\mathbf{z}} \|\mathbf{x} - G_{\theta}(\mathbf{z})\|^2.$$

- But a point mass would have infinite KL divergence. (Exercise: check this.) So the KL term forces q to be more spread out.

Reparameterization Trick (optional)

- To fit q , let's assign it a parametric form, in particular a Gaussian distribution: $q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$, where $\boldsymbol{\mu} = (\mu_1, \dots, \mu_K)$ and $\boldsymbol{\Sigma} = \text{diag}(\sigma_1^2, \dots, \sigma_K^2)$.
- In general, it's hard to differentiate through an expectation. But for Gaussian q , we can apply the **reparameterization trick**:

$$z_i = \mu_i + \sigma_i \epsilon_i,$$

where $\epsilon_i \sim \mathcal{N}(0, 1)$.

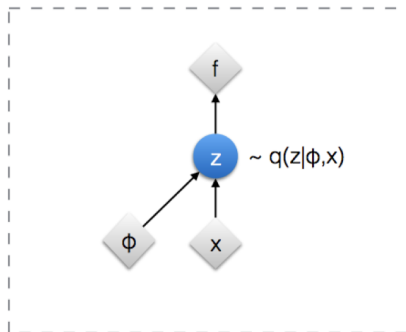
- Hence,

$$\overline{\mu_i} = \overline{z_i} \quad \overline{\sigma_i} = \overline{z_i} \epsilon_i.$$

- This is exactly analogous to how we derived the backprop rules for dropout.

Reparameterization Trick (optional)

Original form



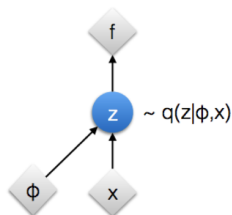
: Deterministic node



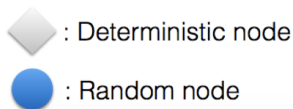
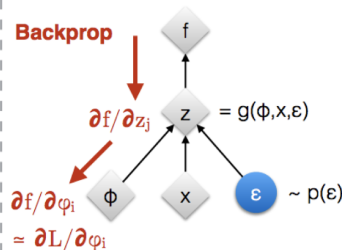
: Random node

Reparameterization Trick (optional)

Original form



Reparameterised form



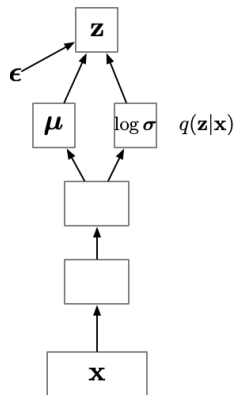
[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

Amortization

- This suggests one strategy for learning the decoder. For each training example,
 - 1 Fit q to approximate the posterior for the current $\mathbf{x}$ by doing many steps of gradient ascent on $\mathcal{F}$.
 - 2 Update the decoder parameters θ with gradient ascent on $\mathcal{F}$.
- **Problem:** this requires an expensive iterative procedure for every training example, so it will take a long time to process the whole training set.

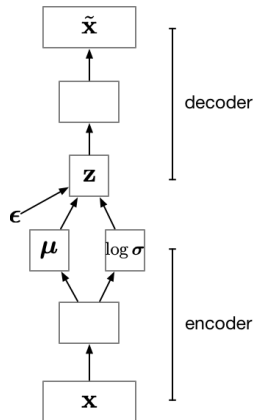
Amortization

- **Idea:** amortize the cost of inference by learning an **inference network** which predicts (μ, Σ) as a function of $\mathbf{x}$.
- The outputs of the inference net are μ and $\log \sigma$. (The log representation ensures $\sigma > 0$.)
- If $\sigma \approx \mathbf{0}$, then this network essentially computes $\mathbf{z}$ deterministically, by way of μ .
 - But the KL term encourages $\sigma > 0$, so in general $\mathbf{z}$ will be noisy.

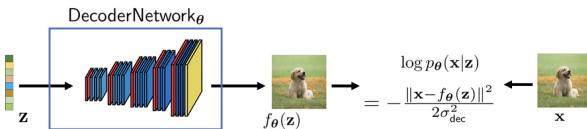
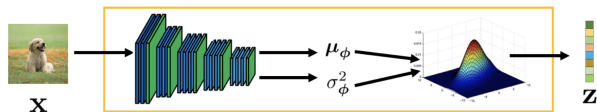


Amortization

- Combining this with the decoder network, we see the structure closely resembles an ordinary autoencoder. The inference net is like an encoder.
- Hence, this architecture is known as a **variational autoencoder (VAE)**.
- The parameters of both the encoder and decoder networks are updated using a single pass of ordinary backprop.
 - The reconstruction term corresponds to squared error $\|\mathbf{x} - \tilde{\mathbf{x}}\|^2$, like in an ordinary VAE.
 - The KL term regularizes the representation by encouraging $\mathbf{z}$ to be more stochastic.



VAEs Visualized



Training VAEs

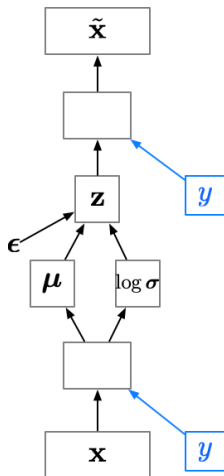
Objective:

$$\begin{aligned} \operatorname{argmax}_{\theta, \phi} \sum_{\mathbf{x} \in \mathcal{X}} \operatorname{ELBO}_{\theta, \phi}(\mathbf{x}) \\ \operatorname{ELBO}_{\theta, \phi}(\mathbf{x}) = -\frac{1}{M} \sum_{m=1}^M \frac{\|\mathbf{x} - G_{\theta}(\mu_{\phi}(\mathbf{x}) + \sigma_{\phi}(\mathbf{x})\epsilon^{(m)})\|^2}{2\sigma_{\text{dec}}^2} \\ + \frac{1}{2} (\sigma_{\phi}(\mathbf{x})d - d + \|\mu_{\phi}(\mathbf{x})\|^2 - 2d \log \sigma_{\phi}(\mathbf{x})) \end{aligned}$$

Typically in training only take one sample, or even just set $\epsilon = 0$

Class-Conditional VAE

- So far, we haven't used the labels y . A **class-conditional VAE** provides the labels to both the encoder and the decoder.
- Since the latent code z no longer has to model the image category, it can focus on modeling the stylistic features.
- If we're lucky, this lets us **disentangle** style and content. (Note: disentanglement is still a dark art.)
- See Kingma et al., "Semi-supervised learning with deep generative models."



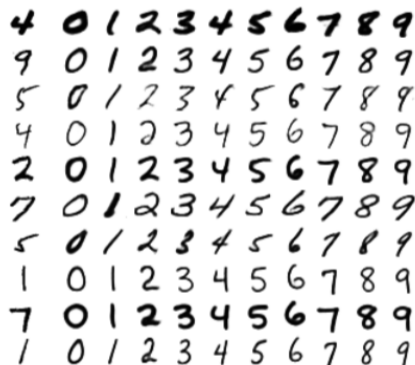
Class-Conditional VAE

- By varying two **latent dimensions** (i.e. dimensions of $\mathbf{z}$) while holding y fixed, we can visualize the **latent space**.



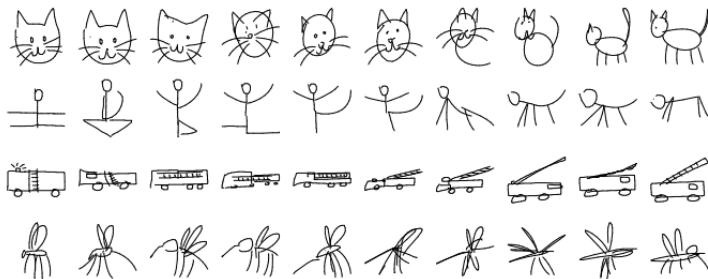
Class-Conditional VAE

- By varying the label y while holding z fixed, we can solve image analogies.



Latent Space Interpolations

- You can often get interesting results by interpolating between two vectors in the latent space:



Ha and Eck, "A neural representation of sketch drawings"

Latent Space Interpolations

- Latent space interpolation of music:
<https://magenta.tensorflow.org/music-vae>

Trade-offs of Generative Approaches

- So far, we have seen three different approaches:
 - Autoregressive models
 - Generative adversarial networks
 - Variational autoencoders
- They all have their own pros and cons. We often pick a method based on our application needs.
- Some considerations for computer vision applications:
 - Do we aim to evaluate the log likelihood of new data?
 - Do we prefer good qualitative samples over an evaluation metric?
 - How important is representation learning, i.e., meaningful code vectors?
 - What computational resources do we have access to?

Trade-offs of Generative Approaches

- In summary:

	Log-likelihood	Sample	Representation	Computation
Autoregressive	Tractable	Good	Poor	$O(\text{\#pixels})$
GANs	Intractable	Good	Good	$O(\text{\#layers})$
VAEs (optional)	Tractable*	OK	Good	$O(\text{\#layers})$

- There is no silver bullet in generative modeling.